



ОНЛАЙН-ОБРАЗОВАНИЕ

Проверить включена ли запись?



Меня хорошо слышно && видно?



Напишите в чат, если есть проблемы!

Ставьте ☐ + если все хорошо

Ставьте ☐ - если есть проблемы



Нестерова Екатерина

Ведущий эксперт в компании АО Гринатом

- В отрасли с 2018 года
- В OTUS с прошлого года
- Являюсь преподавателем на курсе “Разработчик на Spring Framework” в OTUS

Продвинутая конфигурация Spring-приложений



- Договоренности прежние
- С ДЗ спешить не надо
- Дедлайн по ним – дата завершения группы
- Круто, когда получается сдать ДЗ с первого раза
- Когда не с первого – еще круче
- Не стесняемся задавать вопросы по замечаниям

- Области действия бинов (Beans scopes)
- Жизненный цикл бинов

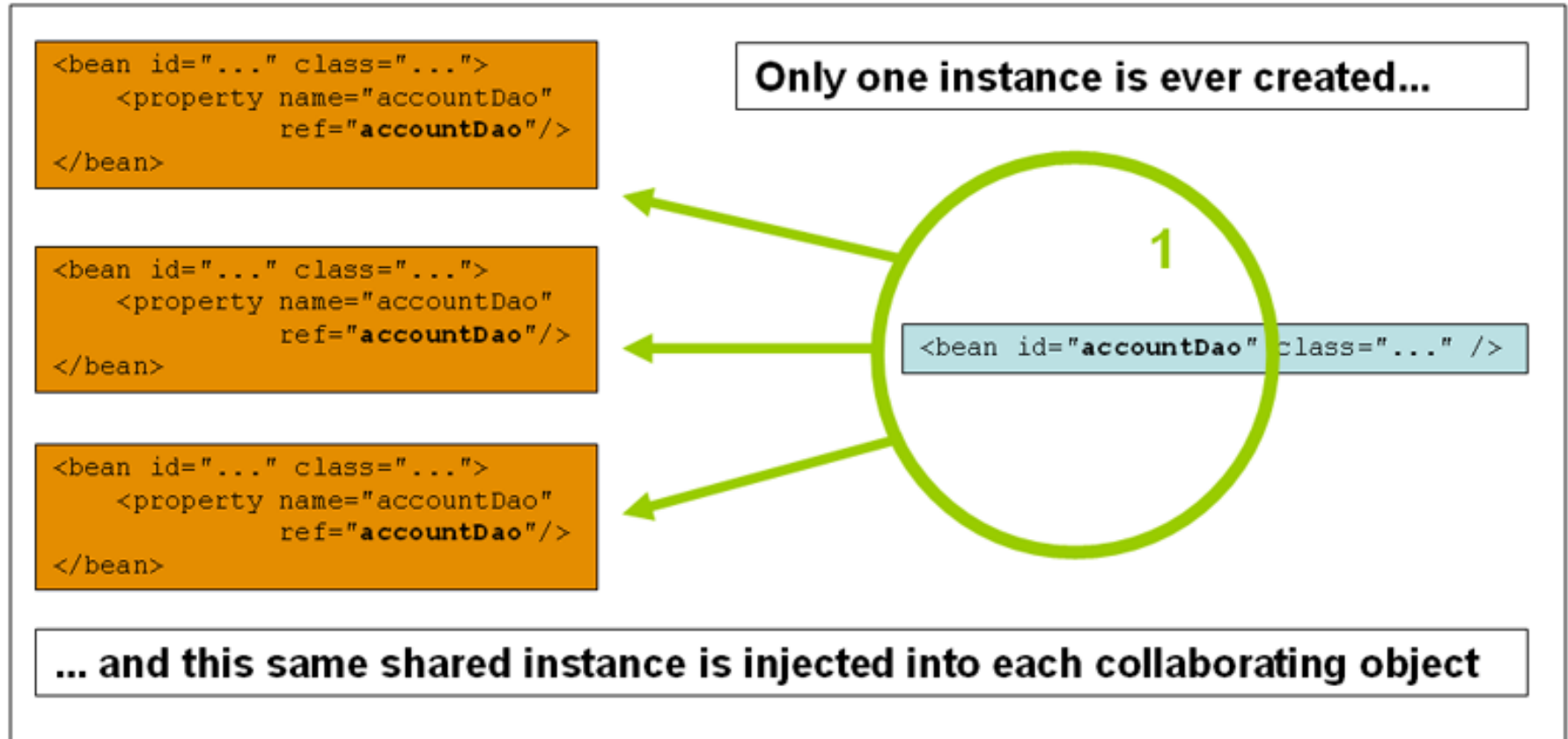
Поехали!

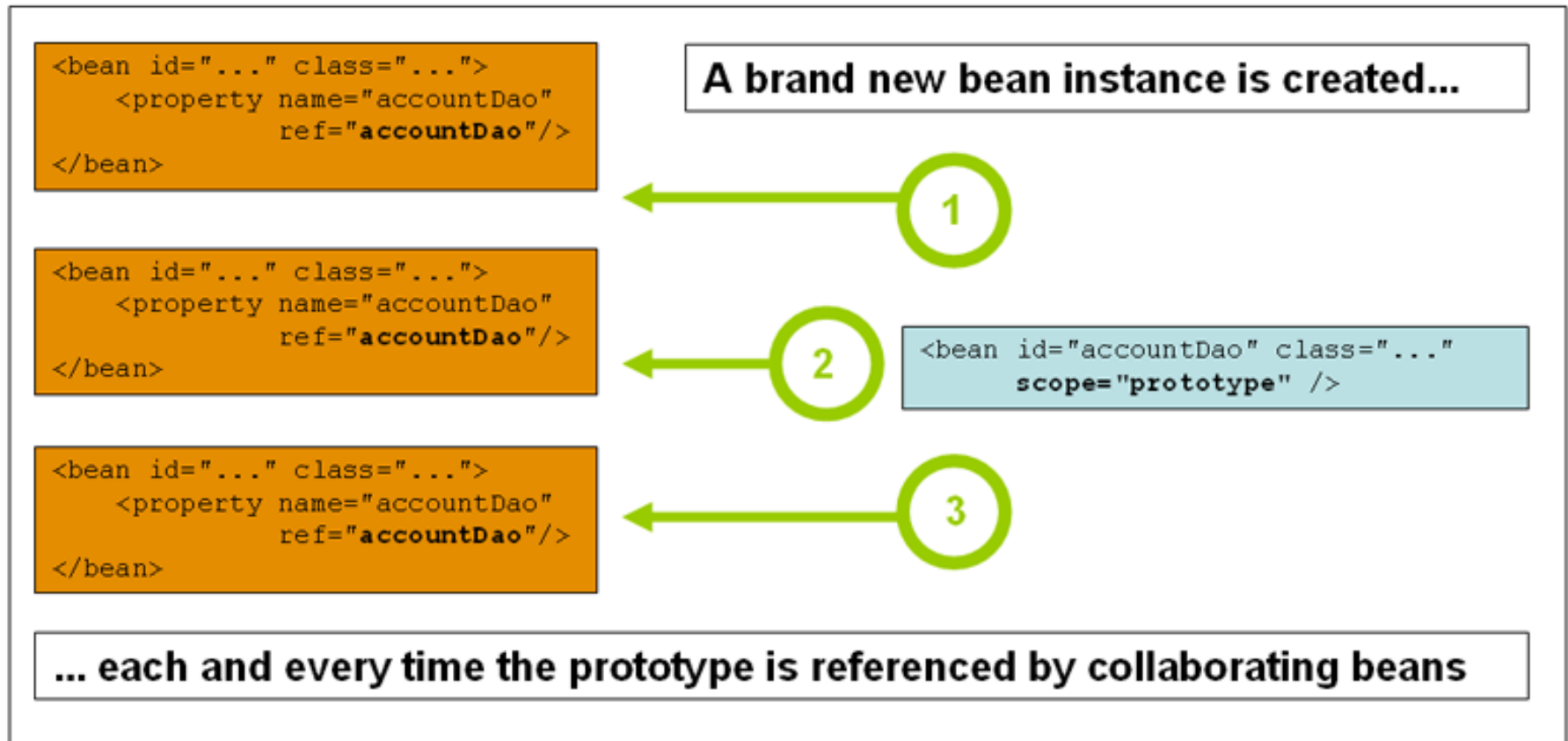
01

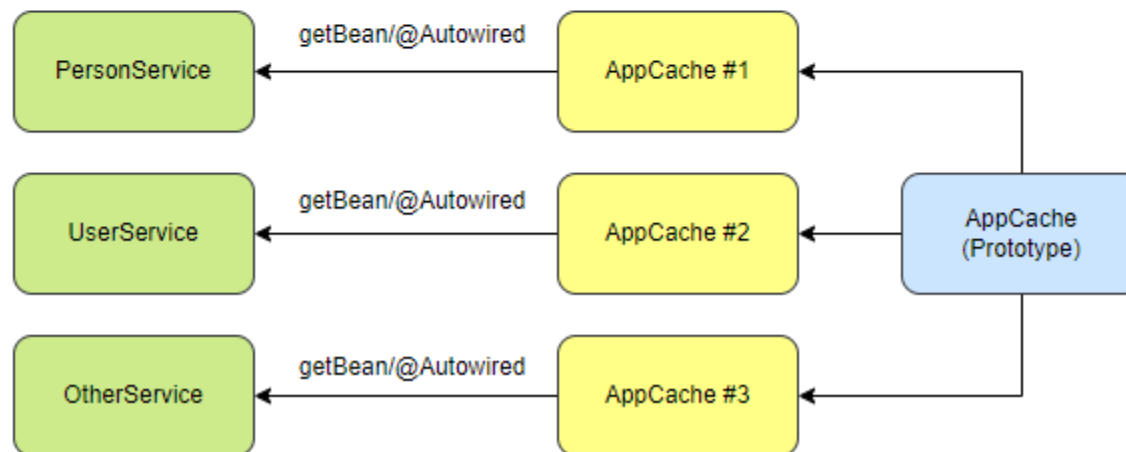
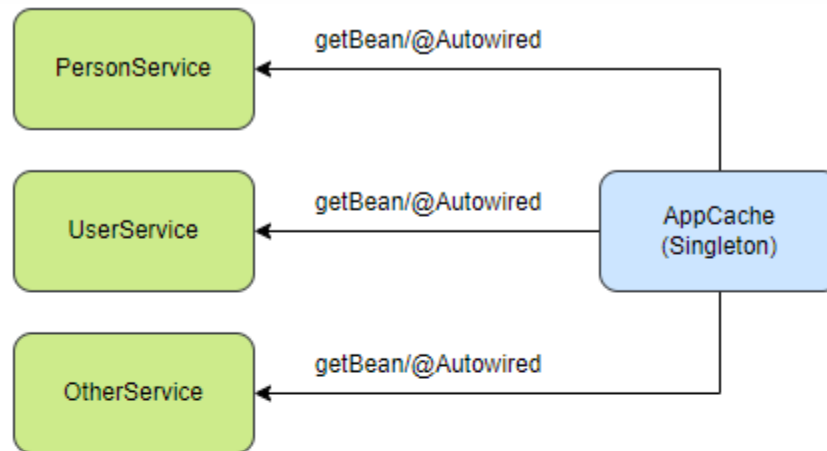
Области действия бинов (Beans scopes)



- Singleton – создаётся один на всё приложение, в зависимости подставляется один объект
- Prototype – объект не создаётся пока не является зависимостью другого бина, в каждый бин инжектится собственный экземпляр
- Request – бин создаётся на каждый Web-запрос и в каждом запросе он свой
- Session – то же, только в рамках Servlet HTTP-сессии
- WebSocket – то же, только в рамках WebSocket - сессии







```
@Scope("session")
@Service
public class AnyServiceImpl implements AnyService {
    ...
}
```

```
@Scope("session")
@Bean
public AnyService anyService () {
    ...
}
```

```
@Scope("session")
```

```
@Bean
```

```
public AnyDependency anyDependency () {  
    return new AnyDependencyImpl();  
}
```

```
@Bean
```

```
public AnyService anyService (AnyDependency dep) {  
    ....  
}
```

```
@Scope(value = "session", proxyMode = ScopedProxyMode.INTERFACES)
@Bean
public AnyDependency anyDependency () {
    return new AnyDependencyImpl();
}

@Bean
public AnyService anyService (AnyDependency dep) {
    ....
}
```



```
@Scope(value = "session", proxyMode = ScopedProxyMode.TARGET_CLASS)
@Bean
public AnyDependency anyDependency () {
    return new AnyDependencyImpl();
}

@Bean
public AnyService anyService (AnyDependency dep) {
    ....
}
```

Упражнение №1

(beans-scopes-exercise)



- ✓ Расставить скоупы над сервисами так, чтобы приложение запустилось и заработало, как указано на странице <http://localhost:8080>
- ✓ Подсказки в названии сервисов
- ✓ Не забываем про proxyMode для некоторых скоупов

```
1. git clone git@github.com:OtusTeam/Spring.git
   или
   git clone https://github.com/OtusTeam/Spring.git

2. cd ./<имя группы>
   /spring-05-bean-scopes-and-lifecycle
   /beans-scopes-exercise
```

Слушатели, которые смотрят нас в записи,
могут поставить паузу.

- Есть разные scope бинов
- Prototype не спасёт от проблем с thread safety
- proxyMode – поможет вставить бины из другого скоупа

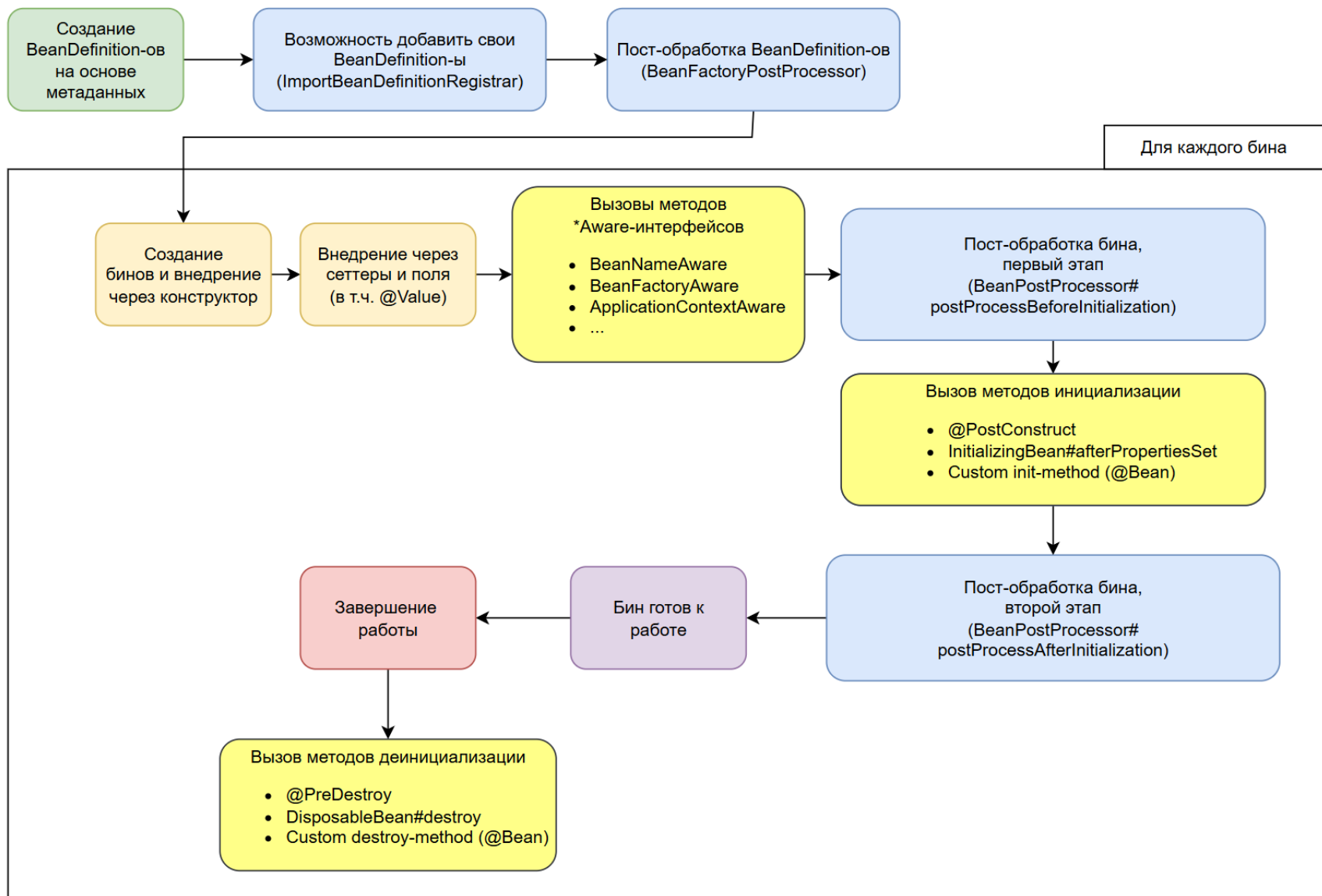
Вопросы?

(ставим “+” если вопросы есть и “-”
если вопросов нет)

02

Жизненный цикл бинов





@Service

```
public class AnyServiceImpl implements InitializingBean {  
    @Override  
    public void afterPropertiesSet() throws Exception {  
        // Do custom initialization  
    }  
}
```

@Service

```
public class AnyServiceImpl {  
    @PostConstruct  
    public void anyMethodName() {  
        // Do custom initialization  
    }  
}
```


@Service

```
public class AnyServiceImpl implements DisposableBean {  
    @Override  
    public void destroy() throws Exception {  
        // Do custom deinitialization  
    }  
}
```

@Service

```
public class AnyServiceImpl {  
    @PreDestroy  
    public void anyMethodName () {  
        // Do custom deinitialization  
    }  
}
```

```
@Configuration
```

```
public class ServiceConfig {
```

```
    @Bean(initMethod = "anyInitMethodName",  
          destroyMethod = "anyDestroyMethodName")
```

```
    public void anyMethodName () {
```

```
        return new AnyServiceImpl();
```

```
    }
```

```
}
```

- О большинстве этапов цикла приходится вспоминать, только когда происходят странные ошибки при инициализации контекста.
- Обычно программист вклинивается в две фазы – `init` и `destroy`

Упражнение №2

(beans-lifecycle-exercise)



- ✓ Изучить классы ничего в них не меняя:
 - CustomBeanDefinitionRegistrar
 - CustomBeanFactoryPostProcessor
 - CustomBeanPostProcessor
 - CustomLifecycleBean
- ✓ Запустить приложение
- ✓ Изучить распечатанный жизненный цикл

```
1. git clone git@github.com:OtusTeam/Spring.git
   или
   git clone https://github.com/OtusTeam/Spring.git

2. cd ./<имя группы>
   /spring-05-bean-scopes-and-lifecycle
   /beans-lifecycle-exercise
```

Слушатели, которые смотрят нас в записи, могут поставить паузу.

Упражнение №3

(beans-lifecycle-exercise)



Упражнение 3.

- ✓ В application.yml выставить lifecycle.print.enabled в false
- ✓ Запустить приложение. Запомнить результат
- ✓ В CustomBeanFactoryPostProcessor раскомментировать блок кода
- ✓ Запустить приложение. Что изменилось?
- ✓ В CustomBeanPostProcessor раскомментировать строку
- ✓ Запустить приложение. Что изменилось теперь?

```
1. git clone git@github.com:OtusTeam/Spring.git
   или
   git clone https://github.com/OtusTeam/Spring.git

2. cd ./<имя группы>
   /spring-05-bean-scopes-and-lifecycle
   /beans-lifecycle-exercise
```

Слушатели, которые смотрят нас в записи, могут поставить паузу.

- Есть сложный жизненный цикл
- Init и destroy пишутся в случае крайней необходимости
- И через @PostConstruct и @PreDestroy
- *Aware – нарушение всех принципов IoC если они используются в бизнес-классах
- BeanFactoryPostProcessors и BeanPostProcessors – позволяют работать с бинами до и во время создания

Вопросы?

(ставим “+” если вопросы есть и “-”
если вопросов нет)



Открытый урок

Score бинов в Spring



[Ссылка с привязкой ко времени, когда рассказ про Отус уже прошел](#)

Пожалуйста, пройдите опрос

Спасибо за внимание!

Жизненного цикла Вам!

